User Defined Channels in occam

Fred Barnes
Computing Laboratory, University of Kent, Canterbury, KENT. CT2 7NF
(frmb2@ukc.ac.uk)

April 25, 2002

Abstract

This technical note describes the *user-defined channels* extension to occam[1] [1], using KRoC [2] on the i386/Linux platform. User-defined channels provide a way of interfacing occam programs to the OS environment using channels and user provided code. The basic idea is that the programmer provides functions, in C, which will perform the necessary channel operations (input, output and alternative), invoked by channel usage.

1 Introduction

User-defined channels are a mechanism for allowing the programmer to interface C code with occam channels, in such a way that the C code gets executed when the channel is involved in input, output or `ALT`ernatives. The occam programmer only needs to be slightly aware that these are special channels, at the point of their declaration. For the most part, they can be used like ordinary channels.

This note describes how to use user-defined channels, it does not attempt to describe their implementation – that will come later.

Sections 3 through 3.3 describe the main programming details for using user-defined channels. This should at least be glanced over before attempting your own things, as there are several important things to note. Section 3.4 describes the additional occam kernel entry-points needed for user-defined channels. This is not required knowledge, but is useful if you plan on doing things in transputer assembly from occam. Section 4 describes what needs to be done in order for occam to use the external channels. Section 5 covers the issues associated with handling I/O code which needs to block in the linux-kernel during an `ALT`.

Sections 6 through 8 present the implementation of three different external channel types. These are included in the KRoC/Linux distribution from 1.3.2-pre2 onwards. The three different implementations vary in complexity, starting with the simplest and finishing with the most complex.

Section 9 describes how non-simple occam protocols are handled. Section 10 describes some thoughts about the sort of uses these can be put to, along with things which I currently consider incomplete or just plain broken.

[1] occam is a trademark of ST Microelectronics

2 Motivation

The occam language has always generally been self-contained with respect to the environment in which it operates. Of course, the transputer had no environment as such, the compiled occam ran on the raw hardware, usually aided with some bootstrap code. In those days, the majority of I/O was performed via a server running on a workstation connected to the transputer (sometimes containing the transputer on a plug-in card), using "hostio" and "hostsp" [3]. KRoC still supports these libraries, but instead of communicating to some server they are wired via glue-code to the relevant C library calls.

Access to the UNIX environment for KRoC is mostly done through the use of David Wood's C interface [4]. This relies on the translator, `tranx86` [5], intercepting the external C calls and generating the code for a C call, rather than an occam call. The blocking system-calls extensions [6] build on this to provide a way of performing a blocking operation without suspending the whole occam run-time system.

While these external interfaces support the majority of needs, there are still some short-comings. One of these is the event driven nature of CSP [7] combined with the non-event driven nature of procedure calls (how external C calls and hostio/hostsp do their work). This makes it particularly hard to `ALT`ernate on an external event, especially one which is likely to block. Fortunately blocking system-calls provide a way to terminate an in-progress call, as demonstrated by the occam socket library [8]. Getting this right is hard however, as can be seen from the example code (in [8]).

The user-defined channels extension attempts to address some of these problems by providing an event-driven channel interface to the external environment, and providing one which is easy for the occam programmer to use.

3 External channels

The implementation of user-defined channels is based around the notion of *external channels* in a modified version of CCSP [9], currently used in the KRoC/Linux system. Unlike external channels as used by Vella [10, 11], which were designed for networking, these allow the user to use their own C code to implement channel communication. External channels also existed in the original CCSP, but these were related largely to the ties with MESH [12], which uses the raw networking hardware (via special device drivers[2] and user-space code) to perform communication.

The implementation of external channels in the modified CCSP is based around a data structure which defines the external interface, some additional functions which enable the programmer to perform certain tasks, and a number of additional kernel entry-points which implement communication on external channels. These three main things are examined in the following sections.

3.1 The `ext_chan_t` structure

The '`ext_chan_t`' structure is defined in one of the CCSP header files, "`udc.h`" and describes a single external channel:

[2] The MESH device drivers are only used for getting access to consecutively physical memory and for memory mapping the hardware into user-space. They don't perform any actual communications themselves.

```
typedef struct {
    // private to CCSP

    void *chanword;
    unsigned int chantype;
    void *chanbxalt;
    void *chansync;

    // user stuff

    unsigned int magic;
    int flags;

    int (*chan_verify)(ext_chan_t *, unsigned int);
    int (*chan_read)(ext_chan_t *, char *, int);
    int (*chan_write)(ext_chan_t *, char *, int);
    int (*chan_alt_enable)(ext_chan_t *);
    int (*chan_alt_disable)(ext_chan_t *);
    int (*chan_min)(ext_chan_t *, void **);
    int (*chan_mout)(ext_chan_t *, void **);
    int (*chan_min64)(ext_chan_t *, unsigned long long *);
    int (*chan_mout64)(ext_chan_t *, unsigned long long *);
    int (*chan_minn)(ext_chan_t *, unsigned int *, int);
    int (*chan_moutn)(ext_chan_t *, unsigned int *, int);

    void *userptr;
} ext_chan_t;
```

The first four elements of this structure are used by CCSP at run-time and should not be used, at all, by the programmer. Rather than allocating it themselves, a function in CCSP is provided (section 3.2), which can also allocate additional space, a pointer to which is left in the '`userptr`' element.

The '`magic`' word should be set the the constant value "`UDC_MAGIC`". This is used for verifying the integrity of external channels during communication. If for some reason damaged user-code jumps on this structure, CCSP will abort sanely, rather than causing a run-time error by following a damaged pointer.

The '`flags`' field allows the programmer to specify things about how the external channel should behave. Currently, the defined flags, which may be `OR`'d together are:

- `UDC_NONE` is the empty flag. This should be used if no flags are needed.
- `UDC_BLOCKING_INPUT` specifies that channel inputs should be handed off to another linux process, on the assumption that they will block in the linux kernel.
- `UDC_BLOCKING_OUTPUT` specifies that channel outputs should be handed off to another linux process.
- `UDC_BLOCKING` specifies that both channel inputs and outputs should be handed off to another linux processes (bitwise `OR` of the previous two flags).
- `UDC_BLOCKING_ALT` specifies that the mechanism used to implement '`ALT`' will block while waiting. This is described fully in section 5.

The next eleven elements of the `ext_chan_t` structure are pointers to functions which will perform the various aspects of communication. These are described fully in section 3.3. These are also checked for non-NULL pointers if the run-time kernel needs to call one of them, resulting in a run-time error if not defined. The three examples, in sections 6 through 8, all support `ALT`s. Supporting `ALT`s is not entirely trivial however, so the programmer may choose not to implement them. In these cases, attempting to `ALT` on that channel in occam will result in a run-time error.

The final element, '`userptr`' is a pointer for the programmer's use. This would typically be some structure specific to the channel being implemented, but can be used for anything. If requested, the CCSP allocation function for the `ext_chan_t` structure will also allocate extra memory, placing a pointer to it in '`userptr`'.

3.2 User callable functions in CCSP

The CCSP run-time kernel provides a number of C functions to help the programmer use external/user-defined channels. These are proto-typed in the "`udc.h`" header file, which the user C code would normally include. There are five in total:

- `ext_chan_t *ccsp_udc_alloc_extchan (int extra_bytes)`

  This is used to allocate the channel structure. This should be called from a C function which is used to initialise the channel, ultimately providing an address for the occam program (section 4). If '`extra_bytes`' is non-zero, then this many bytes will be allocated as well, with a pointer to it being placed in the returned channel's '`userptr`'.

- `void ccsp_udc_free_extchan (ext_chan_t *chan)`

  This is used to free a channel structure. This should be called from a C function which is used to free and destroy the channel. It should be ensured that this does not happen whilst the channel is in use, or undefined behaviour will result. After the channel has gone out of scope in occam is generally a good place.

- `void ccsp_udc_resume_process (ext_chan_t *chan)`

  This is used to resume a process if it was blocked in the channel. This is different from the kernel-blocking mechanism, which is used if either of the blocking input or output flags are specified for the channel. The '`softchan`' and '`bufchan`' examples (sections 6 and 7) use this channel-blocking mechanism.

- `void ccsp_udc_start_alter (ext_chan_t *chan,`
  `void (*func)(void *, int *), void *arg)`
  `int ccsp_udc_kill_alter (ext_chan_t *chan)`

  These are used to start and kill a kernel-blocking process used during `ALT`s. The use of these is described fully in section 5. The '`sockchan`' example (section 8) uses this to support `ALT`ing on a socket read (implemented using '`select`(2)').

3.3 User I/O functions

The bulk of code for an external channel lives in functions written by the programmer, which should be be pointed at appropriately in the `ext_chan_t` structure, usually by an initialisation function (section 4).

3.3.1 Channel verification

The first function, 'chan_verify' is always required. This is used to verify that the channel is safe to use. In the simplest case, this will be something like:

```
int my_channel_verify (ext_chan_t *chan, unsigned int hashcode)
{
    return 0;
}
```

This code doesn't perform any verification as such, it just returns with zero indicating that the channel is valid. A non-zero return will cause the occam kernel to abort with a run-time error. The 'chan' parameter is the channel being verified and 'hashcode' is a hash-code of the occam channel protocol. The 'hashcode' will generally be used if this mechanism is being used to connect two occam programs through some external channel mechanism – it's wise to check that both ends are speaking the same protocol, otherwise undefined behaviour will result. Some of the time, in 'sockchan' for example (section 8), this is not used.

The include file 'udc.h' contains a number of pre-defined constants for certain values of 'hashcode'. These are intended to be used if the channel needs to carry a particular protocol. The buffered channel example (section 7) demonstrates this. The current set of pre-defined values is:

constant	**channel type**
UDC_CHANTYPE_INT	CHAN OF INT
UDC_CHANTYPE_BYTE	CHAN OF BYTE
UDC_CHANTYPE_BOOL	CHAN OF BOOL
UDC_CHANTYPE_INT_C_BYTE	CHAN OF INT::[]BYTE
UDC_CHANTYPE_P_INT	CHAN OF PROTOCOL (INT)
UDC_CHANTYPE_P_BYTE	CHAN OF PROTOCOL (BYTE)
UDC_CHANTYPE_P_BOOL	CHAN OF PROTOCOL (BOOL)
UDC_CHANTYPE_P_INT_C_BYTE	CHAN OF PROTOCOL (INT::[]BYTE)

3.3.2 Channel reads and writes

The two functions 'chan_read' and 'chan_write' are used to implement the bulk of the communication. In the C world, these would typically take the form:

```
int my_channel_read (ext_chan_t *chan, char *buf, int count)
{
    .. read 'count' bytes from 'chan' into 'buf'
}

int my_channel_write (ext_chan_t *chan, char *buf, int count)
{
    .. write 'count' bytes from 'buf' to 'chan'
}
```

Only one of these need be provided if the channel is going to be used exclusively for input or output. The 'UDC_BLOCKING_INPUT' and 'UDC_BLOCKING_OUTPUT' flags affect how these functions

are invoked, and what their return value means. For plain non-kernel-blocking inputs and outputs, i.e. channels without those flags, the return value indicates whether the occam process that performed the input/output should block in the channel. A return value of zero indicates that the occam process should continue immediately, on the assumption that the input or output was performed successfully. A return value of 1 indicates that the occam process should be suspended in the channel. In the case where the occam process is suspended, 'ccsp_udc_resume_process' can be used by the other side (reader/writer) to wake it up again. The awakened process is removed from the channel and placed on the run-queue, resuming directly in occam.

The programmer is responsible for keeping track of whether something is blocked in the channel or not. Looking at the 'chanword' element in the 'ext_chan_t' structure is not an acceptable thing to do. The 'softchan' example (section 6) demonstrates this, by using the 'userptr' to flag whether something is blocked in the channel. The 'bufchan' example (section 7) goes little further, since is has a private structure attached to its 'userptr'.

For channel inputs or outputs where the 'UDC_BLOCKING_INPUT' or 'UDC_BLOCKING_OUTPUT' flag is specified, the user functions are called from a separate kernel-thread, by using *blocking system-calls* [6] – slightly modified to handle 3 arguments. In this case, the return value has no meaning, the occam process will remain suspended until the user function returns – 'my_channel_read' and 'my_channel_write' in this example. The socket-channel (section 8) does input and output in this way. ALTernating on things which need to block in the linux-kernel is covered in section 5.

3.3.3 Channel ALTernation

The two functions 'chan_alt_enable' and 'chan_alt_disable' are used to implement the alternative sequence. For example, given the occam code:

```
-- c is some user-defined channel
ALT
  c ? x
    .. do something
  other ? y
    .. do something else
```

The 'chan_alt_enable' function is called during the enabling sequence, where the various ALT guards are examined to see if they are ready. The programmer provides enable and disable functions along the lines of:

```
int my_channel_enable (ext_chan_t *chan)
{
    .. work out if channel ready
    return ready;
}

int my_channel_disable (ext_chan_t *chan)
{
    .. work out if channel ready now
    return ready;
}
```

For simple things like the the 'softchan' and 'bufchan' examples (sections 6 and 7), the code used to implement the ALT enable and disable is relatively trivial. The enable code indicates (via 'userptr') that an ALT has started if the channel was not ready. This means the writer has to explicitly check the state of the other side (for ALT or plain read) and do the appropriate thing. The disabling code checks the flag and stops advertising the ALT to writers, which may have been explicitly changed if the writer arrived and modified it (which is the case for 'softchan' and 'bufchan').

When the enabling function, 'my_channel_enable' in the above fragment, returns zero to indicate a non-ready channel, CCSP links the occam process which is ALTing into the channel. When the writer arrives and sees an ALTing reader, it should, if desired, wake up the ALTing process by calling 'ccsp_udc_resume_process' with the channel.

The mechanism for handling ALTs involving things which need to block in the linux-kernel is described in section 5. The structure of these functions remains mostly the same though.

3.3.4 Fixed-size MOBILE communication

The two functions 'chan_min' and 'chan_mout' are used to implement static (fixed-size) MOBILE [13, 14] input and output. For static MOBILEs, space for all 'MOBILE' variables is pre-allocated in *mobile-space* [14], and all internal communication is done by pointer-exchange (semantically the communication is one-way). When compiling with MOBILEs and UDC ('-X5' and '-e' kroc flags currently), additional workspace is allocated beyond the MOBILE pointer to hold the size. This *mobile-size-field* is initialised with the size of the underlying type, and is present in PROC parameters. The pointer-pointer argument passed to the user-function is the workspace address where the MOBILE pointer resides. For example, typical user-code for this might look like:

```
int my_channel_min (ext_chan_t *chan, void **pptr)
{
    int *wsptr = (int *)pptr;
    char *addr = (char *)(wsptr[0]);
    int mobile_size = wsptr[1];

    return my_channel_read (chan, addr, mobile_size);
}

int my_channel_mout (ext_chan_t *chan, void **pptr)
{
    int *wsptr = (int *)pptr;
    char *addr = (char *)(wsptr[0]);
    int mobile_size = wsptr[1];

    return my_channel_write (chan, addr, mobile_size);
}
```

In this example, the functions simply get the needed information from the 'pptr' parameter then call the earlier-defined 'my_channel_(read—write)' functions.

3.3.5 Dynamically-sized MOBILE communication

The four functions 'chan_min64', 'chan_mout64', 'chan_minn' and 'chan_moutn' are used to implement dynamic MOBILE communication. In a similar way to fixed-size MOBILEs, workspace for a *mobile-size-field* is allocated. The current implementation of dynamic MOBILEs allocates a workspace-slot for the data-pointer and a workspace-slot for each dynamic MOBILE dimension. Most dynamic mobiles are 1-dimensional (strings for example), handled through the 'chan_min64' and 'chan_mout64' functions. To these, a 64-bit *unsigned long long* is passed, which indicates the occam workspace address where the pointer and dimension-size live. The multi-dimensional versions (for 2-dimensions and above) are similar, except they pass the workspace address of where the pointer lives and a count indicating the number of dimensions. The dimensions are ordered highest-first in workspace-slots above the pointer, followed finally by the *mobile-size-field*.

Communication of dynamic mobiles is done with a copy, after which the outputting (occam) process sets the first dimension-size to zero (undefined/unallocated). If the UDC needing dynamic-mobile support does it within the same occam system, then this pointer can be copied from the outputting process to the inputting process, along with the dimension-size. For cases where the other-side of the communication is elsewhere or non-dynamic-mobile, the programmer must explicitly allocate and free the memory required. Access to the Brinch-Hansen [15] flavoured heap allocator, used for handling dynamic memory in KRoC is provided through the CCSP header file "dmem_if.h". The two most-commonly used functions (allocate and free) are prototyped with:

```
void *dmem_alloc (int size);
void dmem_release (void *ptr);
```

As an example, the following functions provide dynamic-mobile communication in terms of read and write (twice, so the dimensions get communicated first). The code assumes blocking reads and writes (using blocking system-calls, hence why they always return 0):

```
int my_chan_min64 (ext_chan_t *chan, unsigned long long *ptr)
{
    int *wsptr = (int *)ptr;
    char **addrptr = (char **)wsptr;
    int *dimptr = (int *)(&(wsptr[1]));
    int type_size = wsptr[2];

    /* input dimension */
    my_channel_read (chan, (char *)dimptr, sizeof (int));

    /* allocate buffer to hold data */
    *addrptr = (char *)dmem_alloc ((*dimptr) * type_size);

    /* input data */
    my_channel_read (chan, *addrptr, (*dimptr) * type_size);
    return 0;
}

int my_chan_mout64 (ext_chan_t *chan, unsigned long long *ptr)
{
    int *wsptr = (int *)ptr;
    char *addr = (char *)(wsptr[0]);
```

```
    int *dimptr = (int *)(&(wsptr[1]));
    int type_size = wsptr[2];

    /* output dimension */
    my_channel_write (chan, (char *)dimptr, sizeof (int));

    /* output data */
    my_channel_write (chan, addr, (*dimptr) * type_size);

    /* free buffer */
    dmem_release ((void *)addr);
    return 0;
}
```

The implementation of '`my_chan_min64`' and '`my_chan_mout64`' here could of course be defined in terms of the multi-dimsional versions with:

```
int my_chan_min64 (ext_chan_t *chan, unsigned long long *ptr)
{
    return my_chan_minn (chan, (unsigned int *)ptr, 1);
}

int my_chan_mout64 (ext_chan_t *chan, unsigned long long *ptr)
{
    return my_chan_moutn (chan, (unsigned int *)ptr, 1);
}
```

The implementation of '`my_chan_minn`' and '`my_chan_moutn`' simply being more generic versions of the 64-bit (1-dimensional) functions:

```
int my_chan_minn (ext_chan_t *chan, unsigned int *ptr, int dimcount)
{
    int *wsptr = (int *)ptr;
    char **addrptr = (char **)wsptr;
    int *dimptr = (int *)(&(wsptr[1]));
    int type_size = wsptr[dimcount + 1];
    int n, d;

    /* input dimensions */
    my_channel_read (chan, (char *)dimptr, sizeof (int) * dimcount);

    /* calculate memory required and allocate buffer */
    for (d = 0, n = type_size; d < dimcount; d++, n *= dimptr[d]);
    *addrptr = (char *)dmem_alloc (n);

    /* input data */
    my_channel_read (chan, *addrptr, n);
    return 0;
}
```

```
int my_chan_moutn (ext_chan_t *chan, unsigned int *ptr, int dimcount)
{
    int *wsptr = (int *)ptr;
    char *addr = (char *)(wsptr[0]);
    int *dimptr = (int *)(&(wsptr[1]));
    int type_size = wsptr[dimcount + 1];
    int n, d;

    /* output dimensions */
    my_channel_write (chan, (char *)dimptr, sizeof (int) * dimcount);

    /* calculate memory required and output buffer */
    for (d = 0, n = type_size; d < dimcount; d++, n *= dimptr[d]);
    my_channel_write (chan, addr, n);

    /* free buffer */
    dmem_release ((void *)addr);
    return 0;
}
```

3.4 Additional kernel entry-points

A number of CCSP kernel entry-points have been devoted to supporting user-defined channels. These are the points from which the user-defined code is invoked. The code to invoke these is either generated directly by the occam compiler (occ21), in the form of five new instructions, or by the translator (tranx86) when translating existing communication instructions. The mix is so that the compiler can generate ordinary code for cases where it doesn't know if a channel is external or not, and can generate special code when it does know a channel is external. The occam programmer may, of course, use these extra instructions directly from blocks of transputer-code ('ASM' blocks).

The following sections describe each one briefly.

3.4.1 EXTVRFY – external verify

EXTVRFY #14		(secondary instruction)
regs in	*areg*	type hash-code, may not be zero
	breg	ext_chan_t channel address
regs out	*areg'*	*creg*
	breg'	*undefined*
	creg'	*undefined*

This is used to verify an external channel. The underlying C function pointed at by 'chan_verify' in the 'ext_chan_t' structure, pointed at by *breg*, is called with the type hash-code in *areg*. A run-time error occurs if anything fails during verification.

3.4.2 EXTIN – external input

`EXTIN #60`		(secondary instruction)
regs in	*areg*	count
	breg	`ext_chan_t` channel address
	creg	destination address
regs out	*areg'*	*undefined*
	breg'	*undefined*
	creg'	*undefined*

This is used to perform an input on an external channel. The underlying C function pointed at by '`chan_read`' in the '`ext_chan_t`' structure, pointed at by *breg*, is called to read *areg* bytes into the memory pointed at by *creg*.

3.4.3 EXTOUT – external output

`EXTOUT #61`		(secondary instruction)
regs in	*areg*	count
	breg	`ext_chan_t` channel address
	creg	source address
regs out	*areg'*	*undefined*
	breg'	*undefined*
	creg'	*undefined*

This is used to perform external channel output. The underlying C function pointed at by '`chan_write`' in the '`ext_chan_t`' structure, pointed at by *breg*, is called to write *areg* bytes starting at address *creg*.

3.4.4 EXTENBC – external enable channel

`EXTENBC #66`		(secondary instruction)
regs in	*areg*	guard
	breg	`ext_chan_t` channel address
regs out	*areg'*	guard
	breg'	*creg*
	creg'	*undefined*

This is used to enable an external channel, as part of an alternative sequence. The underlying C function pointed at by '`chan_alt_enable`' in the '`ext_chan_t`' structure, pointed at by *breg*, is called to enable the channel. The the guard in *areg* is false, then the C function will not get called.

3.4.5 EXTNDISC – external disable channel

`EXTNDISC #67`		(secondary instruction)
regs in	*areg*	process offset
	breg	guard
	creg	`ext_chan_t` channel address
regs out	*areg'*	fired
	breg'	*undefined*
	creg'	*undefined*

This is used to disable an external channel, as part of an alternative sequence. The underlying C function pointed at by '`chan_alt_disable`' in the '`ext_chan_t`' structure, pointed at by *creg*, is called the disable the channel. If the guard in *breg* is false, then the C function will not get called. If the user C function returns non-zero, then the process offset in *areg* is placed in *Wptr[Temp_p]*.

3.4.6 EXTMIN – external mobile input

`EXTMIN #68`		(secondary instruction)
regs in	*areg*	`ext_chan_t` channel address
	breg	address of destination mobile pointer
regs out	*areg'*	*undefined*
	breg'	*undefined*
	creg'	*undefined*

This is used to perform a mobile input on an external channel. The underlying C function pointed at by '`chan_min`' in the '`ext_chan_t`' structure, pointed at by *areg*, is called to read a mobile into the memory pointed at by de-referencing *breg*. If *mobile-size-fields* are enabled (default in KRoC when using UDCs), the size of the base-type can be found at '*breg*`[1]`' (slot above the pointer).

3.4.7 EXTMOUT – external mobile output

`EXTMOUT #69`		(secondary instruction)
regs in	*areg*	`ext_chan_t` channel address
	breg	address of source mobile pointer
regs out	*areg'*	*undefined*
	breg'	*undefined*
	creg'	*undefined*

This is used to perform a mobile output on an external channel. The underlying C function pointed at by '`chan_mout`' in the '`ext_chan_t`' structure, pointed at by *areg*, is called to write the mobile pointed at by de-referencing *breg*. If *mobile-size-fields* are enabled (default in KRoC when using UDCs), the size of the base-type can be found at '*breg*`[1]`' (slot above the pointer).

3.4.8 EXTMIN64 – external dynamic mobile input

EXTMIN64 #6A		(secondary instruction)
regs in	*areg*	`ext_chan_t` channel address
	breg	address of destination pointer and dimension
regs out	*areg'*	*undefined*
	breg'	*undefined*
	creg'	*undefined*

This is used to perform a single-dimensional dynamic mobile input from an external channel. The underlying C function pointed at by '`chan_min64`' in the '`ext_chan_t`' structure, pointed at by *areg*, is called to read the data. The data-pointer and dimension should be placed in the two workspace slots pointed at by *breg* (pointer first). If *mobile-size-fields* are enabled (default in KRoC when using UDCs), the size of the base-type can be found at '*breg*`[2]`' (slot above the dimension). If memory needs to be allocated, the '`chan_min64`' function should allocate it using '`dmem_alloc`'.

3.4.9 EXTMOUT – external dynamic mobile output

EXTMOUT64 #6B		(secondary instruction)
regs in	*areg*	`ext_chan_t` channel address
	breg	address of source pointer and dimension
regs out	*areg'*	*undefined*
	breg'	*undefined*
	creg'	*undefined*

This is used to perform a single-dimensional dynamic mobile output on an external channel. The underlying C function pointed at by '`chan_mout64`' in the '`ext_chan_t`' structure, pointed at by *areg*, is called to write the data. The data-pointer and dimension can be found in the two workspace slots pointed at by *breg* (pointer first). If *mobile-size-fields* are enabled (default in KRoC when using UDCs), the size of the base-type can be found at '*breg*`[2]`' (slot above the dimension). If the data is copied rather than *moved*, the '`chan_mout64`' function should free it using '`dmem_release`'.

3.4.10 EXTMINN – external multi-dimensional dynamic mobile input

EXTMINN #6E		(secondary instruction)
regs in	*areg*	dimension count
	breg	`ext_chan_t` channel address
	creg	address of destination pointer and dimensions
regs out	*areg'*	*undefined*
	breg'	*undefined*
	creg'	*undefined*

This is used to perform a multi-dimensional dynamic mobile input from an external channel. The underlying C function pointed at by '`chan_minn`' in the '`ext_chan_t`' structure, pointed

at by *breg*, is called to read data of *areg* dimensions. The data-pointer and dimensions should be placed in the workspace slots pointed at by *breg* (pointer first, then the 1st dimension, 2nd dimension, ...). If *mobile-size-fields* are enabled (default in KRoC when using UDCs), the size of the base-type can be found at '*breg*`[`*areg* `+ 1]`' (slot above the innermost dimension). If memory needs to be allocated, the '`chan_minn`' function should allocate it using '`dmem_alloc`'.

3.4.11 EXTMOUTN – external multi-dimensional dynamic mobile output

`EXTMOUTN #6F`		(secondary instruction)
regs in	*areg*	dimension count
	breg	`ext_chan_t` channel address
	creg	address of source pointer and dimensions
regs out	*areg'*	*undefined*
	breg'	*undefined*
	creg'	*undefined*

This is used to perform a multi-dimensional dynamic mobile output on an external channel. The underlying function pointed at by '`chan_moutn`' in the '`ext_chan_t`' structure, pointed at by *breg*', is called to write data of *areg* dimensions. The data-pointer and dimensions can be found in the workspace slots pointed at by *breg* (pointer first, then the 1st dimension, 2nd dimension, ...). If *mobile-size-fields* are enabled (default in KRoC when using UDCs), the size of the base-type can be found at '*breg*`[`*areg* `+ 1]`' (slot above the innermost dimension). The size of the data in bytes (B) can be calculated by:

$$\mathsf{B} = \prod_{i=1}^{\mathcal{A}^{reg}+1} \mathcal{B}^{reg}_{[i]} \tag{1}$$

That being the product of all the dimensions and the base-type size. If the data is copied rather than *moved*, the '`chan_moutn`' function should free it using '`dmem_release`'.

4 The occam interface

This section describes the work needed in occam to use a user-defined channel. There are two main parts to this, firstly allocating and initialising the '`ext_chan_t`' structure, then secondly, binding it to an occam channel. The version of KRoC/Linux which supports this work also includes the three examples (sections 6 through 8), and some example occam programs which use them, as part of the distribution. These can be found in the "`udc/libsrc/`" and "`udc/examples/`" directories.

In order to allocate the `ext_chan_t` structure, a function in the C world must be provided, with an occam interface to call it. The parameters to these functions will depend largely on the nature of the user-defined channel being used. The three example user-defined channels can be accessed through "`udc.inc`" and "`udc.lib`", which define some occam `PROC`s to do the necessary:

```
PROC udc.softchan.alloc (RESULT INT addr)
PROC udc.softchan.free (INT addr)
PROC udc.bufchan.alloc (VAL INT size, RESULT INT addr)
PROC udc.bufchan.free (INT addr)
```

```
PROC udc.sockchan.alloc (VAL []BYTE hostname, VAL INT port, RESULT INT addr)
PROC udc.sockchan.free (INT addr)
```

These simply wrap-up C functions with the same parameters. An occam program would call one of these to get an address for a user-defined channel, then bind it to an occam channel. For example:

```
INT addr:
SEQ
  -- new buffered channel please
  udc.bufchan.alloc (16, addr)

  -- bind to channel and do process network
  PLACED CHAN INT c addr:
  PAR
    producer (c!)
    consumer (c?)

  -- free channel
  udc.bufchan.free (addr)
```

This is all the interaction the occam program needs to see. Inside the scope of the 'PLACED' channel, 'c' is used just like any other 'CHAN INT'. The code above makes use of the optional 'OF' and 'AT', as well as channel-direction specifiers. This bit might traditionally be written:

```
-- bind to channel and do process network
PLACED CHAN OF INT c AT addr:
PAR
  producer (c)
  consumer (c)
```

Using channel-direction specifiers is recommended however, since it makes the code clearer and allows the compiler to perform various additional checks on channel usage. The occam compiler generates the EXTVRFY instruction at the point the channel is PLACED.

5 Blocking channel ALTs

The handling of kernel-blocking inputs in ALTs is non-trivial. Effort has been made however to make supporting ALTs as easy as possible. The principle is that the ALT enabling function dispatches a blocking-system call which will wait for a ready-state. The enabler returns immediately to continue enabling the other guards. The dispatching of this call is handled by the CCSP function:

```
void ccsp_udc_start_alter (ext_chan_t *chan,
                           void (*func)(void *, int *), void *arg)
```

The programmer provides a function which performs the actual blocking, of the form:

```
void my_chan_alter (void *arg, int *wakeflag)
{
    ... wait for event

    *wakeflag = 1;
    return;
}
```

The '`arg`' passed is the same as that passed to the '`ccsp_udc_start_alter`' command, which would normally be the `ext_chan_t` pointer. However, different cases may require different things, so a generic (`void *`) pointer is used.

'`waitflag`' is a pointer to a special flag which is used to to wake up the `ALT`ing process. Initially this flag is zero. When the blocking call terminates, either because the event happened or the call was cancelled, this flag is examined. If set to 1, the `ALT`ing process will be resumed if it is still waiting. Any other value will cause the `ALT`ing process to be left alone.

As mentioned, the blocking call can either finish normally when it gets the event, or it is terminated by the channel disabling sequence. CCSP provides the following function to kill a blocked `ALT`er:

```
int ccsp_udc_kill_alter (ext_chan_t *chan)
```

This interfaces to the blocking system-calls '`C.killcall`' [6], which will terminate a blocked call. It returns one of the following values:

return value	**meaning**
-1	the call already finished normally
0	the call has been signalled for termination
≥ 1	serious problem

When -1 is returned, the channel should be considered ready, since the blocking call finished. Anything else should be considered not-ready. Zero is the only likely 'anything' case here, other values should be impossible, but it might be worth checking if things appear to be going wrong.

It is important that the user-code keeps track of whether the event occurred or not, such as not to try and terminate it if it has already finished, presumably because the event it was waiting for happened. However, in such cases, things will still work properly, but the saving of overheads makes it worth avoiding.

At a slightly higher level, the `UDC_BLOCKING_ALT` flag is used to control termination of the blocking call. If set, a synchronisation occurs between the exiting blocking call (regardless of whether it was terminated or exited normally) and the channel-disabling sequence, after the user C channel disabling function has returned. The synchronisation is one way in that the channel-disable will wait for the blocking `ALT`er to finish, but not visa-versa. This flag must be used on the channel if performing the enabling sequence is not thread-safe with the blocking `ALT`er function – usually because they would both access some flag describing the state of the `ALT`. The socket channel example (section 8) uses these blocking `ALT`s.

6 softchan – a simple channel

This section shows the implementation of a simple user-defined channel. This one behaves in exactly the same way as a regular soft channel. Firstly, there are some standard headers which one would generally use:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

#include "udc.h"
#include "spunixhdr.h"
```

The 'udc.h' and 'spunixhdr.h' header files are located in the CCSP source tree, symblinked into the user-defined channels directory ('udc/libsrc/' in the distribution).

6.1 Allocation and initialisation

The first main piece of code is the function which allocates and initialises a soft channel, plus the occam interface to it:

```
void real_udc_softchan_alloc (int *addr)
{
    ext_chan_t *chan = ccsp_udc_alloc_extchan (0);

    chan->magic = UDC_MAGIC;
    chan->flags = UDC_NONE;

    chan->chan_verify = udc_softchan_verify;
    chan->chan_read = udc_softchan_read;
    chan->chan_write = udc_softchan_write;
    chan->chan_alt_enable = udc_softchan_enable;
    chan->chan_alt_disable = udc_softchan_disable;
    chan->userptr = NULL;

    *addr = (int)chan;
}

// PROC C.udc.softchan.alloc (RESULT INT addr)
void _udc_softchan_alloc (int *ws)
{
    real_udc_softchan_alloc ((int *)(ws[0]));
}
```

This simply sets up the various things inside the ext_chan_t structure, after having allocated it with 'ccsp_udc_alloc_extchan'. A corresponding function is used to free the channel after it as finished being used:

```
void real_udc_softchan_free (int *addr)
{
    ext_chan_t *chan = (ext_chan_t *)(*addr);
```

```
    ccsp_udc_free_extchan (chan);
    *addr = 0;
}

// PROC C.udc.softchan.free (INT addr)
void _udc_softchan_free (int *ws)
{
    real_udc_softchan_free ((int *)(ws[0]));
}
```

Since these channels are purely internal and can carry any protocol, the verify function simply returns zero (success):

```
int udc_softchan_verify (ext_chan_t *chan, unsigned int hashcode)
{
    return 0;
}
```

6.2 Input and output

The input and output functions simply have to wait for each other, then perform the transfer of data. The '`userptr`' field of the channel structure is used to indicate whether there is anything waiting in the channel, including an `ALT`. Only the writer need check explicitly for a waiting `ALT`. The code for the channel read is as follows:

```
int udc_softchan_read (ext_chan_t *chan, char *ptr, int bytes)
{
    if (!chan->userptr) {
        // channel empty

        chan->userptr = ptr;
        return 1;               // block process in channel
    } else {
        // writer waiting, copy data and wake it up

        memcpy (ptr, chan->userptr, bytes);
        chan->userptr = NULL;              // clear channel
        ccsp_udc_resume_process (chan);
        return 0;            // process resumes immediately
    }
}
```

The code for the writer is largely similar to this, but with an additional check for a waiting `ALT`. This is flagged by the '`userptr`' field pointing back at the channel, rather than a data-buffer if a reader was waiting. The code is as follows:

```
int udc_softchan_write (ext_chan_t *chan, char *ptr, int bytes)
{
    if (!chan->userptr) {
        // channel empty

        chan->userptr = ptr;
```

```
        return 1;               // block process in channel
    } else if (chan->userptr == chan) {
        // ALTing on the other side

        ccsp_udc_resume_process (chan);
        chan->userptr = ptr;
        return 1;               // block process in channel
    } else {
        // reader waiting, copy data and wake it up

        memcpy (chan->userptr, ptr, bytes);
        chan->userptr = NULL;              // clear channel
        ccsp_udc_resume_process (chan);
        return 0;            // process resumes immediately
    }
}
```

6.3 Channel ALTs

The code for handling channel ALTs on this type of channel is relatively trivial. The enabling function is:

```
int udc_softchan_enable (ext_chan_t *chan)
{
    if (chan->userptr) {
        // writer waiting

        return 1;                          // channel ready
    } else {
        // flag that we're ALTing

        chan->userptr = chan;
        return 0;                      // channel not ready
    }
}
```

When executed, the disabling code will either find that the writer arrived, woke the ALT and put itself in the channel, or that something else woke up the ALT and 'userptr' is still pointing at the channel. The code for this is:

```
int udc_softchan_disable (ext_chan_t *chan)
{
    if (chan->userptr == chan) {
        // still flagged as ALTing

        chan->userptr = NULL;
        return 0;                      // channel not ready
    } else {
        // writer arrived and is now blocked in channel

        return 1;                          // channel ready
    }
}
```

7 bufchan – a buffered channel

The buffered channel is slightly more complex than the soft channel in the previous section, and only supports 'INT' channels. As before, a common set of header files are used:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

#include "udc.h"
#include "spunixhdr.h"
```

There is also a private structure which holds the buffer and buffer state:

```
typedef struct {
    int size, head, tail, count;
    int *databuf;
    char *waiting;
} udc_bufchan_t;
```

7.1 Allocation and initialisation

The allocation function for the buffered channel takes a pointer of where to store the channel address and a size for the buffer:

```
void real_udc_bufchan_alloc (int count, int *addr)
{
    ext_chan_t *chan = ccsp_udc_alloc_extchan (sizeof (udc_bufchan_t));
    udc_bufchan_t *priv = (udc_bufchan_t *)chan->userptr;

    // check for a sensible size

    if (count <= 0) {
        fprintf (stderr, "invalid buffer size.\n");
        ccsp_udc_free_extchan (chan);
        call_occam_exit ();
    }

    // initialise channel

    chan->magic = UDC_MAGIC;
    chan->flags = UDC_NONE;

    chan->chan_verify = udc_bufchan_verify;
    chan->chan_read = udc_bufchan_read;
    chan->chan_write = udc_bufchan_write;
    chan->chan_alt_enable = udc_bufchan_enable;
    chan->chan_alt_disable = udc_bufchan_disable;

    // allocate and initialise buffer

    priv->databuf = (int *)malloc (priv->size * sizeof (int));
```

```
    if (!priv->databuf) {
        fprintf (stderr, "out of memory!\n");
        ccsp_udc_free_extchan (chan);
        call_occam_exit ();
    }

    priv->size = count;
    priv->head = 0;
    priv->tail = 0;
    priv->count = 0;
    priv->waiting = NULL;

    // store pointer and return

    *addr = (int)chan;
    return;
}

// PROC C.udc.bufchan.alloc (VAL INT size, RESULT INT addr)
void _udc_bufchan_alloc (int *ws)
{
    real_udc_bufchan_alloc ((int)(ws[0]), (int *)(ws[1]));
}
```

The corresponding function to free the channel is:

```
void real_udc_bufchan_free (int *addr)
{
    ext_chan_t *chan = (ext_chan_t *)(*addr);
    udc_bufchan_t *priv = (udc_bufchan_t *)chan->userptr;

    free (priv->databuf);
    priv->databuf = NULL;
    ccsp_udc_free_extchan (chan);
    *addr = 0;
    return;
}

// PROC C.udc.bufchan.free (INT addr)
void _udc_bufchan_free (int *ws)
{
    real_udc_bufchan_free ((int *)(ws[0]));
}
```

The verification function is implemented to check that the channel is of type `INT`, although extending it to other (even arbitrary) types would be relatively easy:

```
int udc_bufchan_verify (ext_chan_t *chan, unsigned int hashcode)
{
    return (hashcode == UDC_CHANTYPE_INT) ? 0 : 1;
}
```

If this ever returns non-zero, then the CCSP run-time system will abort.

7.2 Input and output

Handling input and output is slightly more complicated with buffered channels, since data must be copied in and out of the buffer. The code for the channel read is:

```
int udc_bufchan_read (ext_chan_t *chan, char *ptr, int bytes)
{
    udc_bufchan_t *priv = (udc_bufchan_t *)chan->userptr;

    if (!priv->count) {
        // buffer empty, block

        priv->waiting = ptr;
        return 1;                           // do block
    }
    if (priv->waiting) {
        // writer waiting, buffer must be full
        // copy data out-of and in-to buffer

        *(int *)ptr = priv->databuf[priv->tail];
        priv->tail++;
        priv->tail %= priv->size;
        priv->databuf[priv->head] = *(int *)priv->waiting;
        priv->head++;
        priv->head %= priv->size;

        // clear channel and wake writer

        priv->waiting = NULL;
        ccsp_udc_resume_process (chan);
        return 0;                         // don't block
    } else {
        // read something from the buffer

        *(int *)ptr = priv->databuf[priv->tail];
        priv->tail++;
        priv->tail %= priv->size;
        priv->count--;
        return 0;                         // don't block
    }
}
```

This is mostly buffer manipulation. If the buffer is empty, the reader blocks in the channel, leaving a pointer to its data buffer in '`priv->waiting`'. The corresponding channel write code, including support for handling an `ALT`ing reader, is:

```
int udc_bufchan_write (ext_chan_t *chan, char *ptr, int bytes)
{
    udc_bufchan_t *priv = (udc_bufchan_t *)chan->userptr;

    if (priv->count == priv->size) {
        // buffer full, block

        priv->waiting = ptr;
        return 1;
```

```
        }
        if (priv->waiting == (char *)priv) {
            // ALT waiting, buffer must be empty, wake it up

            priv->waiting = NULL;
            ccsp_udc_resume_process (chan);
        } else if (priv->waiting) {
            // reader waiting, buffer must be empty, wake it up

            *(int *)priv->waiting = *(int *)ptr;
            priv->waiting = NULL;
            ccsp_udc_resume_process (chan);
            return 0;                           // don't block
        }

        // place data in buffer

        priv->databuf[priv->head] = *(int *)ptr;
        priv->head++;
        priv->head %= priv->size;
        priv->count++;
        return 0;                               // don't block
    }
```

7.3 Channel ALTs

Much like the simple soft channel, the enabling and disabling code for the buffered channel is relatively trivial:

```
    int udc_bufchan_enable (ext_chan_t *chan)
    {
        udc_bufchan_t *priv = (udc_bufchan_t *)chan->userptr;

        if (priv->count || priv->waiting) {
            // channel ready

            return 1;
        } else {
            // channel not ready, flag as ALTing

            priv->waiting = (char *)priv;
            return 0;
        }
    }
```

and:

```
    int udc_bufchan_disable (ext_chan_t *chan)
    {
        udc_bufchan_t *priv = (udc_bufchan_t *)chan->userptr;

        if (priv->waiting == (char *)priv) {
            // still us, remove from channel
```

```
        priv->waiting = NULL;
        return 0;
    } else {
        // writer arrived

        return 1;
    }
}
```

8 sockchan – a socket channel

The socket channel is the most complicated example. It implements a channel which interfaces directly with a TCP/IP socket, additionally supporting `ALT`ing reads, using the '`select`(2)' system call to wait for a *data-available* state. Unlike the other two channels, this one does not attempt to enforce any synchronisation between readers and writers, largely because the two ends aren't necessarily connected. This makes it possible to read and write from the channel sequentially, without instantly deadlocking, although in such cases the TCP buffers can fill and it may deadlock as a result (this behaviour has been observed from the example program, which does sequential I/O on the socket, when connected to the local TCP "echo" port (7)).

The code for this has significantly more headers and a bigger private data structure than either of '`softchan`' or '`bufchan`':

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <fcntl.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <netdb.h>
#include <string.h>
#include <errno.h>

#include "udc.h"
#include "spunixhdr.h"

// ensure this is defined, it should be
#ifndef MAXHOSTNAMELEN
#define MAXHOSTNAMELEN 128
#endif
```

The private data-structure is defined as follows:

```
typedef struct {
    int fd;
    char host[MAXHOSTNAMELEN];
    struct sockaddr_in sin;
    int altflag;
} udc_sockchan_t;
```

'`fd`' is used to hold the file-descriptor for the socket, '`host`' contains a local copy of the remote

host name, 'sin' is used to hold information for the 'connect(2)' system call and 'altflag' is a private flag used to indicate the state of an ALT.

8.1 Allocation and initialisation

The initialisation function for the socket channel is responsible for allocating and initialising the structures needed, as well as opening the socket and connecting to the remote host. In the 'udc.lib' library, an external C call is used, so the occam kernel will block until the connection is made. It is perfectly safe however to use external blocking system calls ('B.' and 'BX.'), for the allocation function – however, no guarantees are made about the thread-safe nature of C library calls, you have to ensure this yourself.

The initialisation code for the socket channel is:

```
void real_udc_sockchan_alloc (char *hname, int hlen, int port, int *addr)
{
    ext_chan_t *chan = ccsp_udc_alloc_extchan (sizeof (udc_sockchan_t));
    udc_sockchan_t *priv = (udc_sockchan_t *)chan->priv;
    struct hostent *hp;

    // check for sane arguments

    if (!hlen || (port < 0)) {
        fprintf (stderr, "bad hostname or port!\n");
        ccsp_udc_free_extchan (chan);
        call_occam_exit ();
    }

    // initialise channel

    chan->magic = UDC_MAGIC;
    chan->flags = UDC_BLOCKING | UDC_BLOCKING_ALT;

    chan->chan_verify = udc_sockchan_verify;
    chan->chan_read = udc_sockchan_read;
    chan->chan_write = udc_sockchan_write;
    chan->chan_alt_enable = udc_sockchan_enable;
    chan->chan_alt_disable = udc_sockchan_disable;

    // initialise private data

    priv->altflag = 0;
    priv->fd = socket (PF_INET, SOCK_STREAM, IPPROTO_TCP);

    if (priv->fd < 0) {
        // failed to create socket

        fprintf (stderr, "sockchan: unable to create socket: %s\n",
            strerror (errno));
        ccsp_udc_free_extchan (chan);
        call_occam_exit ();
    }

    strncpy (priv->host, hname,
```

```
        (hlen < MAXHOSTNAMELEN) ? hlen : (MAXHOSTNAMELEN - 1));
    hp = gethostbyname (priv->host);

    if (!hp) {
        // failed to lookup host name

        fprintf (stderr, "sockchan: unable to lookup address of %s\n",
            priv->host);
        close (priv->fd);
        ccsp_udc_free_extchan (chan);
        call_occam_exit ();
    }

    priv->sin.sin_family = AF_INET;
    priv->sin.sin_port = htons (port);
    memcpy (&(priv->sin.sin_addr), hp->h_addr, sizeof (priv->sin.sin_addr));

    // attempt connection

    if (connect (priv->fd, (struct sockaddr *)&priv->sin, sizeof (priv->sin)) < 0) {
        // failed to connect

        fprintf (stderr, "sockchan: unable to connect to remote host: %s\n",
            strerror (errno));
        close (priv->fd);
        ccsp_udc_free_extchan (chan);
        call_occam_exit ();
    }

    // set address and return

    *addr = (int)chan;
    return;
}
```

The code used to free a channel is largely similar to the others:

```
void real_udc_sockchan_free (int *addr)
{
    ext_chan_t *chan = (ext_chan_t *)(*addr);
    udc_sockchan_t *priv = (udc_sockchan_t *)chan->userptr;

    close (priv->fd);
    ccsp_udc_free_extchan (chan);
    return;
}
```

To call these from occam, the following interface code is used:

```
// PROC C.udc.sockchan.alloc (VAL []BYTE hostname, VAL INT port, RESULT INT addr)
void _udc_sockchan_alloc (int *ws)
{
    real_udc_sockchan_alloc ((char *)(ws[0]), (int)(ws[1]),
        (int)(ws[2]), (int *)(ws[3]));
}
```

```
// PROC C.udc.sockchan.free (INT addr)
void _udc_sockchan_free (int *ws)
{
    real_udc_sockchan_free ((int *)(ws[0]));
}
```

8.2 Input and output

The code used to read and write from a channel is flagged to block in the linux-kernel – both input and output blocking flags are combined in '`UDC_BLOCKING`'. As mentioned before, this has a few implications:

- the return value from the C functions used to read-from and write-to the channel is meaningless (and ignored).
- the C functions may not attempt to resume anything with '`ccsp_udc_resume_process`', in the same way that they can't specify "block" with the return value.
- commitment to input or output is total – the communication must happen. If an errors occurs during communication, the desired action is to block forever in the channel and abort the blocking call (with the '`_exit`(2)' system call, maybe after printing an appropriate error message). This prevents the occam process being rescheduled.

The code used to implement a blocking read is:

```
int udc_sockchan_read (ext_chan_t *chan, char *ptr, int len)
{
    udc_sockchan_t *priv = (udc_sockchan_t *)chan->userptr;
    int got = 0;
    int r;

    while (got < len) {
        r = read (priv->fd, ptr + got, len - got);

        if (!r) {
            // EOF from remote party
            fprintf (stderr, "sockchan: unexpected EOF in udc_sockchan_read.\n");

            // exits the blocking clone, leaving the \occam\ process trapped
            _exit (0);
        } else if (r < 0) {
            // error during read
            fprintf (stderr, "sockchan: read error from socket: %s\n",
                strerror (errno));
            _exit (0);
        }
        got += r;
    }
    return 0;
}
```

Most of this code is concerned with either reading the desired number of bytes, or aborting with an error message, leaving the occam process blocked in the channel input. The writer code is similar:

```
int udc_sockchan_write (ext_chan_t *chan, char *ptr, int len)
{
    udc_sockchan_t *priv = (udc_sockchan_t *)chan->userptr;
    int gone = 0;
    int r;

    while (gone < len) {
        r = write (priv->fd, ptr + gone, len - gone);

        if (r < 0) {
            // error during write

            fprintf (stderr, "sockchan: write error from socket: %s\n",
                strerror (errno));
            _exit (0);
        } else if (!r) {
            // should not happen!

            fprintf (stderr, "sockchan: write returned 0!\n");
            _exit (0);
        }
        gone += r;
    }
    return 0;
}
```

8.3 Channel ALTs

As has been noted previously, the handling of a kernel-blocking 'ALT' is not entirely trivial. The following code forms the alternative enable for the socket channel:

```
int udc_sockchan_enable (ext_chan_t *chan)
{
    udc_sockchan_t *priv = (udc_sockchan_t *)chan->userptr;
    struct timeval tv = {0, 0};
    fd_set readset;
    int i;

    // first, poll socket with select() to see if it's ready
    FD_ZERO (&readset);
    FD_SET (priv->fd, &readset);
    i = select (priv->fd + 1, &readset, NULL, NULL, &tv);

    if (i < 0) {
        // error, socket probably disconnected at the other side

        fprintf (stderr, "sockchan: unable to poll socket: %s\n",
            strerror (errno));
        call_occam_exit ();
    }
```

```
    if (i) {
        // socket ready for reading

        priv->altflag = -1;             // flag ready
        return 1;                       // channel ready
    }
    priv->altflag = 0;                  // flag waiting

    // dispatch blocking ALT call
    ccsp_udc_start_alter (chan, udc_sockchan, (int *)chan);

    return 0;                           // channel not ready
}
```

This function first polls the socket with '`select`(2)' to see if any data is available for reading. If so, it sets '`priv->altflag`' to -1, indicating that the channel is ready and that no blocking call has been dispatched. Otherwise it sets '`priv->altflag`' to 0 and dispatches the blocking `ALT`er, the function '`udc_sockchan_alter`' in this case, which is implemented as:

```
void udc_bsyscall_alter (void *arg, int *wakeflag)
{
    ext_chan_t *chan = (ext_chan_t *)arg;
    udc_sockchan_t *priv = (udc_sockchan_t *)chan->userptr;
    fd_set readset;

    // wait for data ready
    FD_ZERO (&readset);
    FD_SET (priv->fd, &readset);
    if (select (priv->fd + 1, &readset, NULL, NULL, NULL) != 1) {
        // should never happen, indicates an error

        fprintf (stderr, "sockchan: select returned <> 1 in alter!\n");
        _exit (0);
    }

    // socket has data ready
    priv->altflag = 1;

    // flag ALT for waking-up
    *waitflag = 1;
    return;
}
```

The alternative disable for the socket channel is where the main magic happens. Either the blocking call was never dispatched, in which case the channel is ready (-1 in '`priv->altflag`'), or the blocking '`ALT`'er was dispatched, which may or may not have terminated – decided by 0 or 1 in '`priv->altflag`'. The code is:

```
int udc_sockchan_disable (ext_chan_t *chan)
{
    udc_sockchan_t *priv = (udc_sockchan_t *)chan->userptr;

    switch (priv->altflag) {
    case -1:
    case 1:
```

```
        // ready at enable, or became ready after wait

        return 1;                       // channel ready

    case 0:
        // still waiting, kill

        if (ccsp_udc_kill_alter (chan) < 0) {
            // finished since we looked at 'altflag'
            return 1;
        }
        return 0;                       // channel not ready

    default:
        // serious error, something got damaged

        fprintf (stderr, "sockchan: all gone wrong!\n");
        call_occam_exit ();
        return 42;                      // keep compiler happy
    }
}
```

The use of the 'UDC_BLOCKING_ALT' flag during channel allocation causes the occam kernel to synchronise the channel disable with the terminating ALTer. This only happens if the disable instruction returns 0 indicating that the channel is not ready, in which case a blocking ALTer is assumed to have been started in the enable and killed here.

9 Notes on channel protocols

For simple channel protocols, such as 'INT', 'BYTE', known-sized array and user-defined (RECORD) types, the way in which communication works is simple – the relevant underlying C function gets called with a pointer and a size (in bytes). For handling counted-array protocols ('INT::[]BYTE'), sequential protocols ('INT; INT; BOOL') and tagged protocols, the compiler transforms the communication into a sequence of simpler communications.

9.1 Counted-array protocols

For counted-array protocols, a communication of the form:

```
PROTOCOL PROT IS INT::[]BYTE:
CHAN OF PROT c:

c ! 6::"Hello!"
```

gets transformed into:

```
SEQ
  c ! 6 (INT)              -- output 1 word (4 bytes)
  c ! "Hello!" ([]BYTE)    -- output 6 bytes
```

A similar transformation happens to the inputting process, where:

```
c ? len::array
```

becomes:

```
SEQ
  c ? len (INT)            -- input 1 word (4 bytes)
  c ? array ([]BYTE)       -- input "len" bytes
```

9.2 Sequential protocols

For sequential protocols, a communication of the form:

```
PROTOCOL S.PROT IS INT; INT; BOOL:
CHAN S.PROT c:

c ! 42; (MOSTPOS INT); FALSE
```

gets transformed into:

```
SEQ
  c ! 42 (INT)             -- output 1 word (4 bytes)
  c ! (MOSTPOS INT) (INT)  -- output 1 word (4 bytes)
  c ! FALSE (BOOL)         -- output BOOL (4 bytes)
```

The inputting process undergoes the same transformation, but with inputs instead of outputs.

9.3 Tagged protocols

Tagged protocols undergo quite a heavy transformation in the compiler. This is mainly to handle the different tags and to ensure that the inputting process implemented the tag being outputted. For an output of the form:

```
PROTOCOL T.PROT
  CASE
    empty
    some.int; INT
:
CHAN T.PROT c:

SEQ
  c ! empty
  c ! some.int; 42
```

the following transformation occurs:

```
SEQ
  SEQ
    c ! empty (BYTE)       -- output 1 byte (tag value)
  SEQ
    c ! some.int (BYTE)    -- output 1 byte (tag value)
    c ! 42 (INT)           -- output 1 word (4 bytes)
```

The inputting process is allowed to make two different types of input. Firstly a tagged input, which only accepts a single tag value, or a `CASE` input which allows different tags to be used. In the tagged input case, an input of the form:

```
c ? CASE empty
```

gets transformed into the following code:

```
BYTE tag:
SEQ
  c ? tag (BYTE)           -- input 1 byte (tag value)
  IF
    tag = empty
      SKIP                 -- protocol inputs here
  -- causes STOP if not "empty" tag
```

The other type of input, `CASE` input, is written in the form:

```
c ? CASE
  empty
    .. do P
  INT v:
  some.int; v
    .. do Q
```

this gets transformed into the following:

```
BYTE tag:
SEQ
  c ? tag (BYTE)           -- input 1 byte (tag value)
  CASE tag
    empty
      .. do P
    some.int
      INT v:
      SEQ
        c ? v (INT)        -- input 1 word (4 bytes)
        .. do Q
```

It is useful to be aware of these transformations if the user-defined channel being implemented needs to perform special handling for some purpose. For example, the socket-channel might be extended to send a protocol of '`INT::[]BYTE`' over the network as one big single communication, rather than two. This would need the the outputting C function to not output the first time round, but make a note of the count, then perform the actual communication on the second call.

10 Future work

The current status of this work is that everything described here works as far as I think it should. In terms of deficiencies, there is no support for the extended rendezvous [16]. Support for this will be added at some point.

There may also be several bugs in what already exists. I've not fully tested the different combinations of things involving `PLACED` channels in occam, so it's possible that some placements will not compile. A simple channel and integer variable should always work though. I'm pretty convinced that as long as the programmer uses the blocking `ALT` correctly, there will be no strange deadlocks or non-waking up of `ALT`s. The code for '`sockchan`' makes a good basis for implementing your own.

10.1 New occam infrastructure

One of the main things still needing to be done in occam is the wrapping of the channel allocation and free calls in something a little nicer. The current thoughts are for a placed type declaration. For the buffered channel, this would be something like:

```
PLACED TYPE BUFFERED (VAL INT n)
  ALLOC addr IS udc.bufchan.alloc (n, addr):
  FREE addr IS udc.bufchan.free (addr):
:
```

The '`addr`' here is used to tell the compiler which parameter in the call contains the address. The name '`addr`' is treated specially and does not need to be declared.

For the occam programmer, using this interface is simple and intuitive:

```
BUFFERED (5) CHAN INT c:
PAR
  producer (c!)
  consumer (c?)
```

In effect now, the only new the thing the occam programmer uses is the "`BUFFERED (5)`" bit on the channel declaration. The syntax also extends quite cleanly to other user-defined channel types, such as the TCP socket channel, which would be interfaced by:

```
PLACED TYPE SOCKET (VAL []BYTE hostname, VAL INT port)
  ALLOC addr IS udc.sockchan.alloc (hostname, port, addr):
  FREE addr IS udc.bufchan.free (addr):
:
```

Socket channels would then be declared and used with something like:

```
SOCKET ("localhost", 7) CHAN BYTE c:
PAR
  read.from.socket (c?, ...)
  write.to.socket (c!, ...)
```

As it stands though, this interface is currently unimplemented, requiring the occam programmer to use the '`udc.sockchan.alloc()`', etc. calls as shown in section 4.

10.2 New applications

This work opens up a new set of possibilities for ways in which occam programs can interact with the OS environment. The traditional occam programmer is not expected to be writing these for themselves, as that requires some programming knowledge of C and UNIX. Instead, occam programmers are expected to use user-defined channel types provided for them in libraries.

The original motivation for this work came about from the desire to interface an X11 toolkit with occam, which proved to be difficult using the existing mechanisms. The user-defined channels extension should make this task much simpler, since it can provide a link from an occam channel to the various X11/toolkit function calls (via some user-code). The main problem with handling X11 previously was waiting for an event in a parallel thread, without introducing data races with Xlib calls in the main occam thread. The blocking `ALT`er support solves this problem, for example:

```
PROC main.bit ([]CHAN GRAPHIC.REQ in?, []CHAN EVENTS out!)

  GRAPHICS.SERVER (...) CHAN GRAPHIC.PROTO disp.server:
  SEQ
    ...
    WHILE TRUE
      PRI ALT
        --{{{  handle graphics event
        disp.server ? CASE
          ...
            out[i] ! ...
        --}}}
        --{{{  handle application requests
        ALT i = 0 FOR SIZE in
          in[i] ? CASE
            ...
              disp.server ! ...
        --}}}
:
```

In real life, it would probably be useful to have different protocols running either way on the '`disp_server`' channel. However, the point is that whatever lives on the other side of '`disp.server`' is thread-safe with respect to itself (providing the '`UDC_BLOCKING_ALT`' flag is set if needed).

In addition to graphics, implementing networked occam infrastructures becomes simpler, along with various other aspects of OS integration.

Concluding remarks

It is hoped that people will find these user-defined channels a useful addition to the KRoC/Linux system. Many thanks to EPSRC and the UKC Computing Laboratory for providing funding for

this work. Also thanks to Peter Welch and Mario Schweigler for their comments and suggestions on this work.

References

[1] Inmos Limited. occam 2.1 Reference Manual. Technical report, Inmos Limited, May 1995. Available at:
`http://wotug.ukc.ac.uk/parallel/occam/documentation/`.

[2] P.H. Welch and D.C. Wood. The Kent Retargetable occam Compiler. In Brian O'Neill, editor, *Parallel Processing Developments, Proceedings of WoTUG 19*, Concurrent Systems Engineering, pages 143–166. World occam and Transputer User Group, IOS Press, Netherlands, March 1996. ISBN: 90-5199-261-0.

[3] SGS-Thompson Microelectronics Limited. *T9000 occam 2 Toolset Handbook*. SGS-Thompson Microelectronics Limited, 1994. Document number: 72 TDS 457 00.

[4] David C. Wood. KRoC – Calling C Functions from occam. Technical report, Computing Laboratory, University of Kent at Canterbury, August 1998.

[5] F.R.M. Barnes. tranx86 – an Optimising ETC to IA32 Translator. In Majid Mirmehdi Alan Chalmers and Henk Muller, editors, *Communicating Process Architectures 2001*, Concurrent Systems Engineering, pages 265–282, Amsterdam, The Netherlands, September 2001. WoTUG, IOS Press.

[6] F.R.M. Barnes. Blocking System Calls in KRoC/Linux. In P.H.Welch and A.W.P.Bakkers, editors, *Communicating Process Architectures*, volume 58 of *Concurrent Systems Engineering*, pages 155–178, Amsterdam, the Netherlands, September 2000. WoTUG, IOS Press.

[7] C.A.R. Hoare. *Communicating Sequential Processes*. Prentice-Hall, 1985.

[8] Fred Barnes. *Socket, File and Process Libraries for occam*. Computing Laboratory, University of Kent at Canterbury, June 2000. Available at:
`http://www.cs.ukc.ac.uk/people/rpg/frmb2/documents/`.

[9] J.Moores. CCSP – a Portable CSP-based Run-time System Supporting C and occam. In B.M.Cook, editor, *Architectures, Languages and Techniques for Concurrent Systems*, volume 57 of *Concurrent Systems Engineering series*, pages 147–168, Amsterdam, the Netherlands, April 1999. WoTUG, IOS Press.

[10] Kevin Vella. *Seamless Parallel Computing on Heterogeneous Networks of Multiprocessor Workstations*. PhD thesis, The University of Kent at Canterbury, Canterbury, Kent. CT2 7NF, December 1998.

[11] K.Vella and P.H.Welch. CSP/occam on Shared Memory Multiprocessor Workstations. In B.M.Cook, editor, *Architectures, Languages and Techniques for Concurrent Systems*, volume 57 of *Concurrent Systems Engineering Series*, pages 87–119, Amsterdam, the Netherlands, April 1999. WoTUG, IOS Press.

[12] R.W. Dobinson M. Boosten and P.D.V. van der Stok. Fine-Grain Parallel Processing on Commodity Platforms. In *WoTUG 22*, volume 57 of *Concurrent Systems Engineering*, pages 263–276. IOS Press, April 1999.

[13] F.R.M. Barnes and P.H. Welch. Mobile Data Types for Communicating Processes. In *Proceedings of the 2001 International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA'2001)*, volume 1, pages 20–26. CSREA press, June 2001.

[14] F.R.M. Barnes and P.H. Welch. Mobile Data, Dynamic Allocation and Zero Aliasing: an occam Experiment. In Majid Mirmehdi Alan Chalmers and Henk Muller, editors, *Communicating Process Architectures 2001*, Concurrent Systems Engineering, pages 243–264, Amsterdam, The Netherlands, September 2001. WoTUG, IOS Press.

[15] Per Brinch Hansen. Efficient Parallel Recursion. volume 30 of *ACM SIGPLAN notices*, December 1995.

[16] F.R.M. Barnes. Various extensions to the occam compiler, 2001. Available at: `http://www.cs.ukc.ac.uk/projects/ofa/kroc/occ21-extensions.html`.